DG INTERNS HUB

Advanced Job Portal
Week 11 Project Report

Prepared by: Pavan Kumar Katkuri

Submission date: 05 November 2026

Evidence note: This report describes the included companion implementation. Replace pending measurements and verify it against the actual Week 10 application before final submission.

1. Introduction

The Advanced Job Portal is a web application concept that connects job seekers with recruiters. Candidates can register, browse opportunities and submit applications. Recruiters can create job posts. Week 11 focuses on production readiness: deployment architecture, environment configuration, authentication, API security, performance and testing.

2. Project Objective

The objective is to prepare a maintainable application with separate frontend and backend layers. This package demonstrates registration, login, role-aware access, job listing, search, pagination and application submission. It also includes a Postman collection, project report and deployment checklist.

3. Existing Application

The Week 10 project was described as an Advanced Job Portal with authentication, dashboards, API integration, search, filtering and pagination. The original Week 10 source ZIP was not attached during package creation. Therefore, this is a companion starter rather than a verified modification of that source. Confirm feature parity and merge existing dashboards.

4. Deployment Architecture

The recommended architecture hosts the frontend as static assets and the backend as a Node.js API. The browser sends HTTPS requests to the API. The API validates input, checks authentication and role permissions, applies rate limits and accesses persistent storage. Use a managed database in production.

5. Frontend Deployment

The frontend folder contains responsive HTML, CSS and JavaScript. Deploy it to static hosting such as Netlify or Vercel. Configure the API base URL before deployment. Test forms, job search, pagination, navigation and mobile widths. Never place private credentials in frontend JavaScript.

6. Backend Deployment

The backend uses Express. Install dependencies with npm install and start using npm start. On a Node hosting provider, configure the build/start commands, environment variables, HTTPS and health checks. Monitor logs and avoid local file persistence in multi-instance production.

7. Database Deployment

The included JSON store is for local demonstration only. It does not provide transaction safety, concurrent write handling, managed backups or database indexing. Replace it with PostgreSQL or MongoDB before production. Add unique email constraints and indexes for common job searches.

8. Environment Variables

The .env.example file lists PORT, NODE_ENV, JWT_SECRET, FRONTEND_URL, JWT_EXPIRES_IN and ADMIN_EMAIL. Copy it to .env locally and replace sample values. Use a long random JWT secret and provider-managed secrets. The gitignore excludes .env and dependencies. Never commit real credentials.

9. Authentication Security

Passwords are hashed using bcrypt before storage. Login compares the entered password with the stored hash. Successful login returns a signed JWT with an expiry. Protected routes require a Bearer token. Public registration permits candidate and recruiter roles only; admin creation is not exposed.

10. API Security

Helmet adds common HTTP security headers. CORS permits configured frontend origins. The JSON body size is limited. Rate limiting is applied globally and more strictly to login. Express-validator checks registration and job fields. Authorization middleware restricts job creation and applications by role.

11. CORS, Rate Limiting & Input Validation

CORS uses FRONTEND_URL, which must match the deployed frontend origin exactly. General traffic is limited to 100 requests per 15 minutes per IP; login attempts are limited to 10 in the same period. Inputs have length and format checks. Tune limits based on real traffic.

12. Performance Optimization

Job listing supports page and limit parameters and caps page size. The frontend fetches data asynchronously and uses responsive CSS. For production, add database indexes, caching where suitable, compressed assets and image optimization. Measure Lighthouse and API timings against the deployed application.

13. Security Testing & API Testing

Import the Postman collection. Test health, registration, login, job listing, authenticated profile and application submission. Also test invalid email, weak password, duplicate registration, missing/expired token, unauthorized role, oversized input, disallowed CORS origin and repeated login attempts. Save actual response screenshots.

14. Performance & Security Comparison

The following values are intentionally not fabricated. Record actual results from your own deployed project. Use Lighthouse for page performance, Postman for API timings and browser DevTools Network for security response headers.

Area	Before	After / measured evidence
Page loading	Not measured	Not measured
API response	Not measured	Not measured
Lighthouse score	Not measured	Not measured
Security headers	Verify baseline	Verify deployed response
Pagination	Check Week 10	Enabled in sample API
Rate limiting	Check Week 10	Enabled in sample API

15. Screenshots, Challenges & Conclusion

Capture at least 12 genuine screenshots: homepage, login, registration, job listing, search/filter, candidate dashboard, recruiter dashboard, admin dashboard, application workflow, Postman test, security configuration and Lighthouse result. The companion starter does not yet implement all three dashboards; merge those from Week 10. Main learning areas include environment configuration, CORS, JWT, validation, rate limiting, API testing and deployment.